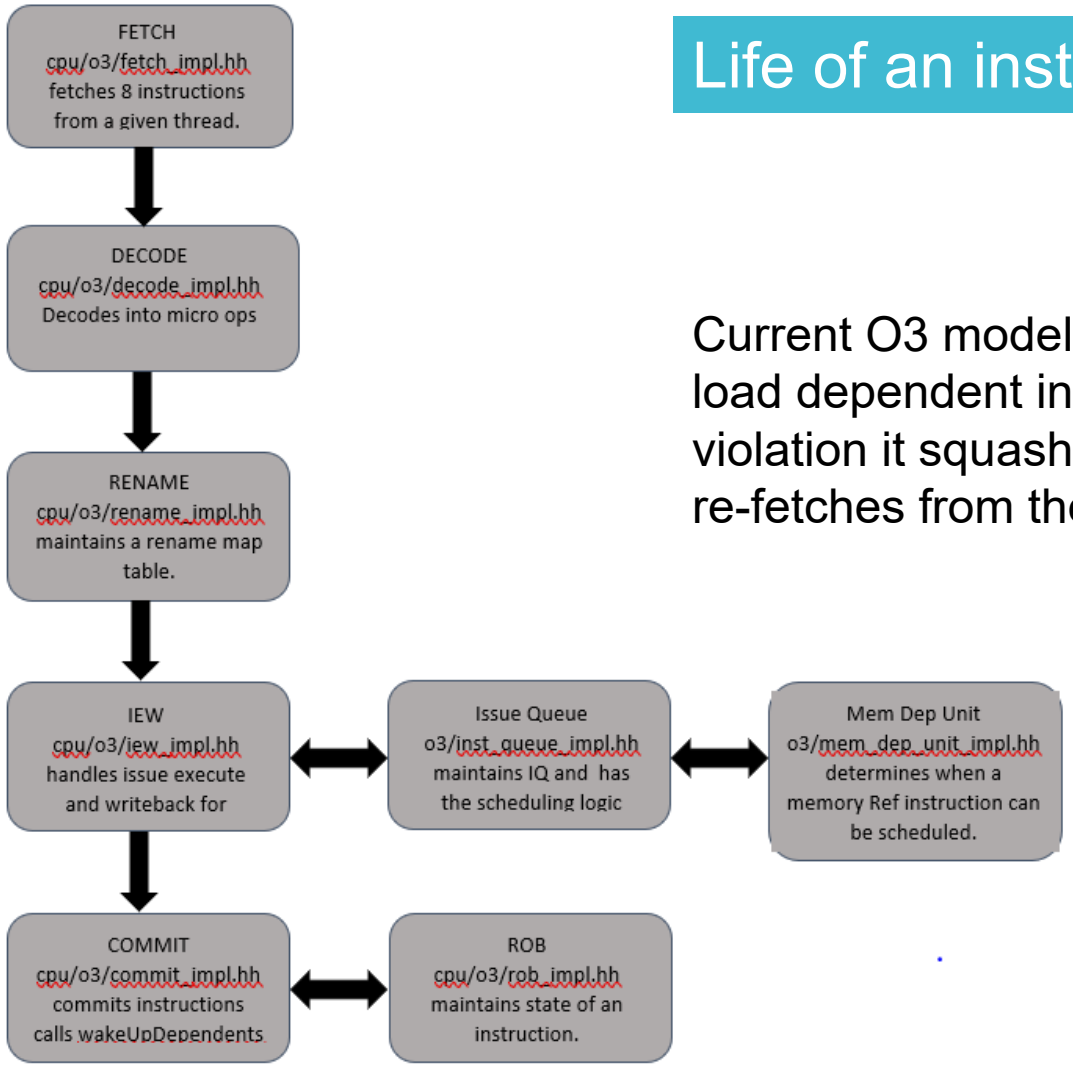
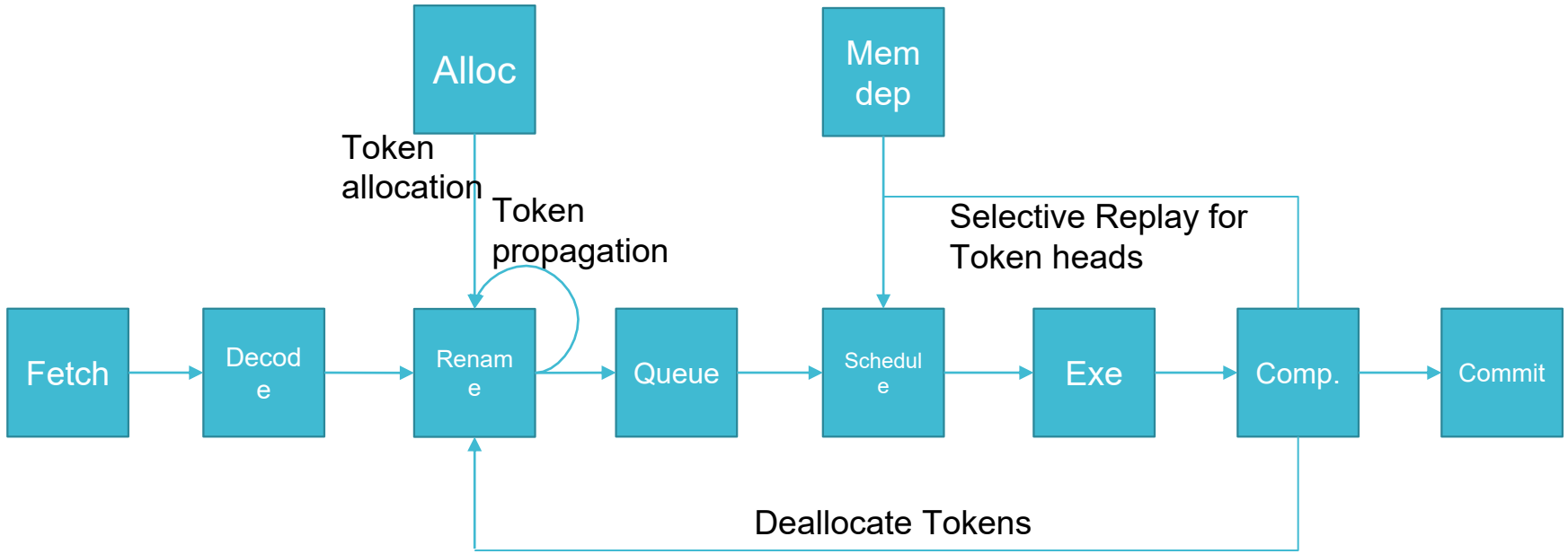
Life of an instruction in the O3 CPU



Current O3 model doesn't allow speculative scheduling of load dependent instructions and on a memory order violation it squashes all the instructions in the pipeline and re-fetches from the violating PC

SELECTIVE TOKEN BASED REPLAY

IDEA – Allocate a token to every load and every dependent instruction inherits the token id in a dependence vector



Architected Register	Physical Register	Dependence vector
R2	p2	00000000
R1	p9	00000001
R4	p7	00000001
R3	p4	00000011
R5	P11	00000011

LOAD R1, [R2] → Token ID 1
ADD R4,R1,R2
LOAD R3,[R4] → Token ID 2
SUB R5,R3,R4

While renaming dest regs once mapping is done

```
If (inst->isLoad) {
  depVector[renamedDestReg] = depVector[renamedDestReg]  | ( 1 << (tokenID -1)
}
Iterate(renamedSrcRegs) {
  depVector[renamedDestReg] = dep[renamedDestReg] |
depVector[renamedSrcReg]
}
```

Changes made to O3 source code

- rename_impl.hh, rename.hh -> tokenID allocation/deallocation, dependence vector calculation, clearing dependence vector
- In the inst_queue_impl.hh instead of popping entries from the instList during commit we added a logic to only pop entries with dependence vector as all zeroes. We maintain a replayQueue which has instructions with non-zero Dependence vectors and remove insts when the commit.
- In lsq_unit_impl.hh when we detect a memory order violation we replay->notify(tokenID). And in the next cycle in issue stage we reissue instructions that match the tokenID from replayQueue.
- Changes made to Fetch and decode and ROB to not squash instructions.